Q. Explain The architecture of Web server, client

Ans.

1. Overview of Web Architecture

Web development follows a client–server model, where:

- Client requests data or services
- Server processes requests and sends responses

This communication mainly happens over the HTTP/HTTPS protocol.

Client (Browser) ⇄ Internet ⇄ Web Server ⇄ Application Logic ⇄ Database

2. Web Client Architecture

What is a Web Client?

A web client is usually a web browser (Chrome, Firefox, Edge, Safari).

Key Components of the Client

1. User Interface (UI)
    - Displays web pages
    - Handles user interactions (clicks, forms, scrolling)
2. Browser Rendering Engine
    - Converts HTML, CSS, and JavaScript into visible content
    - Examples:
        - Blink (Chrome, Edge)
        - Gecko (Firefox)
3. JavaScript Engine
    - Executes JavaScript code
    - Examples:
        - V8 (Chrome)
        - SpiderMonkey (Firefox)
4. Networking Layer
    - Sends HTTP requests (GET, POST, PUT, DELETE)
    - Receives HTTP responses

3. Web Server Architecture

What is a Web Server?

A web server handles incoming client requests and sends responses.

Core Components of a Web Server

1. Web Server Software

Handles HTTP requests and static content

- Apache
- Nginx

2. Application Server

Processes business logic

- Node.js (Express, NestJS)
- Java (Spring Boot)
- Python (Django, Flask)
- PHP (Laravel)

3. Request Handler
    - Routes requests to correct services or controllers
    - Implements REST APIs
4. Middleware
    - Authentication
    - Logging
    - Validation
    - Error handling

Q. Explain HTML 5 - Audio Video Tag

Ans.

HTML5 introduced the **<audio>** and **<video>** tags to embed **multimedia content** directly in web pages **without external plugins** (like Flash).

1. <audio> Tag

Definition

The <audio> tag is used to **embed sound content** such as music or audio streams in a web page.

Basic Syntax

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Supported Audio Formats

- MP3 (audio/mpeg)
- OGG (audio/ogg)
- WAV (audio/wav)

2. <video> Tag

Definition

The <video> tag is used to **embed video content** in a web page.

Basic Syntax

```
<video width="640" height="360" controls>
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

Supported Video Formats

- MP4 (video/mp4)
- WebM (video/webm)
- OGG (video/ogg)

Q. Explain Semantic Elements

Ans.

Semantic elements are HTML elements that **clearly describe their meaning and purpose** to both the **browser** and **developers**. They make the structure of a web page **more readable, accessible, and SEO-friendly**.

👉 Example:

<header> clearly means *page header*, while has **no meaning**.

Why Semantic Elements are Important

- 📖 Better code readability
- 🔍 Improves SEO (search engines understand content)
- ♿ Enhances accessibility (screen readers)
- 🧱 Clear page structure
- 🔧 Easier maintenance

Common HTML5 Semantic Elements

1. <header>

Represents introductory content or navigation links.

```
<header>
  <h1>My Website</h1>
  <nav>...</nav>
</header>
```

2. <nav>

Defines navigation links.

```
<nav>
  <a href="#">Home</a>
  <a href="#">About</a>
</nav>
```

3. <section>

Represents a **thematic grouping** of content.

```
<section>
  <h2>Services</h2>
  <p>We provide web development.</p>
</section>
```

4. <article>

Independent, self-contained content.

```
<article>
  <h2>Blog Post</h2>
  <p>This is a post.</p>
</article>
```

8. <figure> and <figcaption>

Used for images, diagrams, or illustrations.

```
<figure>
  <img src="img.jpg" alt="Image">
  <figcaption>Image description</figcaption>
</figure>
```

9. <mark>

Highlights important text.

```
<p>HTML5 is <mark>important</mark>.</p>
```

Q. Explain Canvas and SVG

Ans.

1. Canvas

Definition

The **<canvas>** element is used to draw **2D graphics** on the fly using **JavaScript**. It is **pixel-based**, meaning drawings are rendered as pixels.

Features

- Resolution dependent (pixel based)
- Requires JavaScript for drawing
- Best for **games**, **animations**, **image processing**
- No built-in support for interactivity per object

Example

```
<canvas id="myCanvas" width="200" height="100"></canvas>
```

2. SVG (Scalable Vector Graphics)

Definition

SVG is an **XML-based vector graphic format** used for defining **shapes and images** using HTML-like tags.

Features

- Resolution independent (vector based)
- Each element is part of the DOM
- Easily animated and styled using CSS and JavaScript
- Best for **icons**, **charts**, **logos**

3. Difference Between Canvas and SVG

Feature	Canvas	SVG
Graphics type	Pixel-based	Vector-based
Scalability	Loses quality	No quality loss
DOM access	❌ No	✅ Yes
Performance	Better for complex animations	Better for static images
Interactivity	Manual	Built-in

Conclusion

- **Canvas** is ideal for **high-performance graphics and games**
- **SVG** is best for **scalable, interactive, and reusable graphics**
- Canvas is suitable for dynamic pixel graphics, whereas SVG is preferred for scalable and interactive vector graphics.

Q. Explain Translate, scale, drag drop & API

Ans.

Translate, Scale, Drag & Drop and API in HTML5

HTML5 and CSS3 provide features to create **interactive and dynamic web applications** such as transformations, drag-drop functionality, and APIs.

1. Translate

Translate is a CSS3 transform property used to **move an element** from its current position along the X and Y axes.

Syntax

transform: translate(50px, 100px);

Features

- Moves elements without affecting layout
- Faster than using margins or positioning
- Used in animations and UI effects

2. Scale

Scale is a CSS3 transform property used to **resize an element**.

Syntax

transform: scale(1.5);

Features

- Scales element horizontally and vertically
- Values >1 enlarge, <1 shrink
- Common in hover effects and zooming

3. Drag and Drop

HTML5 **Drag and Drop API** allows users to **drag elements** and **drop them** into a target area using the mouse.

Key Attributes & Events

- draggable="true"
- Events: dragstart, dragover, drop

Example

<img src="img.png" draggable="true">

Uses

- File uploads
- Moving items in UI
- Games and dashboards

4. API (Application Programming Interface)

An **API** allows communication between **software applications** or components.

HTML5 APIs Examples

- Geolocation API
- Canvas API
- Drag and Drop API
- Web Storage API

Advantages

- Enables advanced web features
- Improves user experience
- Reduces server dependency
- Translate and scale improve visual effects, Drag and Drop enhances interactivity.

Q. Explain Architecture of CSS

Ans.

The **architecture of CSS** explains how **CSS rules are organized, processed, and applied** to HTML elements by the browser to control **presentation and layout**.

Components of CSS Architecture

1. HTML Structure

- HTML provides the **content and structure**
- Elements like <p>, , <h1> act as targets for CSS

2. CSS Stylesheets

CSS can be written in three ways:

- **Inline CSS** – inside HTML element
- **Internal CSS** – inside <style> tag
- **External CSS** – separate .css file (recommended)

3. CSS Parser

- Browser reads and parses CSS
- Checks syntax and converts rules into internal format

4. DOM (Document Object Model)

- HTML is converted into a **DOM tree**
- Represents page structure in memory

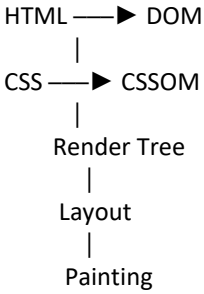
5. CSSOM (CSS Object Model)

- CSS rules are converted into **CSSOM**
- Represents styles as objects

6. Render Tree

- DOM + CSSOM combine to form **Render Tree**
- Contains only visible elements with computed styles

Short Diagram of CSS Architecture



Working of CSS (Step-by-Step)

1. Browser loads HTML and CSS
2. Builds DOM and CSSOM
3. Creates Render Tree
4. Calculates layout
5. Displays styled webpage

Q. Explain SCSS, CSS Modules.

Ans.

Modern web development uses **SCSS** and **CSS Modules** to write **maintainable, scalable, and reusable styles**.

1. SCSS (Sassy CSS)

SCSS is a **CSS preprocessor syntax** of **Sass** that extends CSS with advanced features like variables, nesting, and functions.

Features

- Variables for reuse of values
- Nesting of selectors
- Mixins and functions
- Inheritance using @extend
- Cleaner and modular code

Example

```
$color: blue;

button {
  background: $color;
  &:hover {
    background: darkblue;
  }
}
```

Advantages

- Reduces repetition
- Easy maintenance
- Better code organization

2. CSS Modules

Definition

CSS Modules provide **locally scoped CSS** by default, avoiding global namespace conflicts.

Working

- Each CSS file is treated as a module
- Class names are automatically made unique
- Mostly used in **React, Vue, Angular**

Example

```
/* styles.module.css */
.button {
  color: white;
}
import styles from './styles.module.css';
<button className={styles.button}>Click</button>
```

Advantages

Prevents style conflicts

Improves scalability

Easier debugging

Q. Explain CSS Framework – Bootstrap

Ans.

Bootstrap is a free, open-source CSS framework used to develop responsive and mobile-first web applications quickly and easily. It provides pre-designed CSS classes and JavaScript components.

Features of Bootstrap

1. Responsive Grid System

- Uses a **12-column grid layout**
- Automatically adjusts layout for different screen sizes
- Breakpoints: sm, md, lg, xl

```
<div class="row">
  <div class="col-md-6">Column 1</div>
  <div class="col-md-6">Column 2</div>
</div>
```

2. Predefined CSS Components

- Buttons
- Forms
- Navigation bars
- Cards

```
<button class="btn btn-primary">Submit</button>
```

3. Utility Classes

- Spacing, colors, text alignment, display utilities
- Reduces need for custom CSS

4. JavaScript Components

- Dropdowns
- Carousel
- Tooltip

(Requires JavaScript and Popper.js)

5. Mobile-First Design

- Designed primarily for mobile devices
- Scales up for larger screens

Advantages of Bootstrap

- Saves development time
- Consistent design
- Cross-browser compatible
- Easy to learn
- Large community support

Limitations of Bootstrap

- Websites may look similar
- Larger file size if unused components included
- Customization can be limited

Q. Explain Selectors and Pseudo Classes

Ans.

CSS **selectors** and **pseudo-classes** are used to **select HTML elements** and apply styles based on structure or state.

1. CSS Selectors

A **CSS selector** is used to **target HTML elements** for styling.

Types of CSS Selectors

a) Basic Selectors

- **Element Selector** – selects elements by tag name

```
p { color: blue; }
```

- **Class Selector** – selects elements by class

```
.box { border: 1px solid black; }
```

b) Group Selector

Selects multiple elements at once.

```
h1, h2, p { font-family: Arial; }
```

c) Universal Selector

Selects all elements.

```
* { margin: 0; }
```

2. Pseudo-Classes

Pseudo-classes define the **special state** of an element such as hover, focus, or first child.

Common Pseudo-Classes

Pseudo-Class	Description
:hover	When mouse pointer is over an element
:active	When element is clicked
:focus	When input is focused
:first-child	First child element
:last-child	Last child element
:nth-child()	Selects specific child

Examples

```
a:hover {
  color: red;
}
input:focus {
  border: 2px solid blue;
}
p:first-child {
  font-weight: bold;
}
```

Q. Explain Fonts and Text Effects

Ans. **Fonts in CSS**

Fonts in CSS control the **appearance of text characters** on a web page.

Common font properties:

- font-family – specifies typeface
- font-size – controls text size
- font-style – normal, italic
- font-weight – normal, bold

```
p {
  font-family: Arial;
  font-size: 16px;
  font-weight: bold;
}
```

Text Effects in CSS

Text effects are used to enhance the **visual presentation of text**.

Common text effect properties:

- color – text color
- text-align – alignment (left, center, right)
- text-decoration – underline, overline
- text-transform – uppercase, lowercase

```
h1 {
  text-align: center;
  text-shadow: 2px 2px 4px gray;
}
```

Q. Explain Transition

Ans.

A **CSS Transition** allows an element to **change smoothly** from one style to another over a specified duration instead of changing instantly.

Properties of Transition

- transition-property – CSS property to animate
- transition-duration – time taken for transition
- transition-timing-function – speed curve
- transition-delay – delay before starting

Example

```
button {
  background: blue;
  transition: background 0.5s;
}
button:hover {
  background: red;
```

Q. Explain Overview of responsive web design principles and its significance.

Ans.

Responsive Web Design (RWD) is an approach to web development where a website **automatically adjusts its layout, content, and elements** to fit different screen sizes and devices such as mobiles, tablets, and desktops.

Principles of Responsive Web Design

1. Flexible Grid Layout

- Uses relative units like **percentages** instead of fixed pixels
- Layout resizes proportionally across devices

2. Flexible Images and Media

- Images scale within their containers
- Prevents overflow on small screens

```
img {
  max-width: 100%;
  height: auto;
}
```

3. Media Queries

- Apply different CSS styles based on screen size or device type

```
@media (max-width: 768px) {
  body {
    background: lightgray;
  }
}
```

4. Mobile-First Design

- Design for mobile screens first, then enhance for larger screens
- Improves performance and usability

Significance of Responsive Web Design

- Provides **better user experience** on all devices
- Improves **SEO ranking** (Google prefers mobile-friendly sites)
- Reduces development and maintenance cost
- Increases user engagement
- Ensures accessibility
- Responsive Web Design ensures websites are flexible, user-friendly, and accessible across all devices, making it essential for modern web development.

Q. Explain media queries and viewport meta tag.

Ans.

Media queries are a CSS technique used to apply styles **based on device screen size, resolution, or orientation**. They are a key part of **responsive web design**.

Syntax:

```
@media (max-width: 768px) {
  body {
    background-color: lightgray;
  }
}
```

Uses:

- Adjust layout for mobile, tablet, and desktop
- Improve responsiveness
- Control visibility and styling of elements

Viewport Meta Tag

The **viewport meta tag** controls how a webpage is **displayed on mobile devices**.

Syntax:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Functions:

- Sets page width equal to device width
- Prevents automatic zooming
- Ensures proper scaling on mobile screens
- Media queries control responsive styling, while the viewport meta tag ensures proper scaling on mobile devices.

Q. Explain Techniques for responsive images

Ans.

1. CSS Flexible Images

Use relative sizing so images scale within their containers.

```
img {
  max-width: 100%;
  height: auto;
}
```

2. Media Queries

Serve different image sizes based on screen width.

```
@media (max-width: 600px) {
  .banner {
    background-image: url("small.jpg");
  }
```

Q. Explain srcset, sizes attributes, and picture element.

Ans.

These HTML features are used to create **responsive images**, allowing the browser to load the **most appropriate image** based on screen size and resolution.

1. srcset Attribute

The **srcset** attribute provides a list of image files with different resolutions or widths. The browser automatically selects the best image for the device.

Syntax

```
<img src="image-small.jpg"
    srcset="image-small.jpg 600w,
          image-medium.jpg 1000w,
          image-large.jpg 1600w"
    alt="Image">
```

- 600w, 1000w → image widths
- Useful for different screen resolutions

2. sizes Attribute

The **sizes** attribute tells the browser **how much space the image will take** on the screen under different conditions.

Syntax

```
sizes="(max-width: 600px) 100vw, 50vw"
```

- 100vw → full width on small screens
- 50vw → half width on large screens

3. <picture> Element

Explanation

The **<picture>** element allows **art direction**, meaning different images can be shown based on screen size or format (e.g., mobile vs desktop).

Features

- Uses <source> with media queries
- Fallback <img> is required

Program

```
<!DOCTYPE html>
<html>
<head>
  <title>Responsive Image Demo</title>
</head>
<body>

<h3>Responsive Image Example</h3>

<picture>
  <source media="(max-width: 600px)" srcset="img-small.jpg">
  <source media="(max-width: 1000px)" srcset="img-medium.jpg">
  <img src="img-large.jpg"
     srcset="img-small.jpg 600w,
          img-medium.jpg 1000w,
          img-large.jpg 1600w"
     sizes="(max-width: 600px) 100vw, 50vw"
     alt="Responsive image">
</picture>
</body>
</html>
```

Q. Explain Implementing responsive video and other media.

Ans.

Responsive media ensures that **videos and multimedia elements adapt smoothly** to different screen sizes and devices, improving **usability and performance** in responsive web design.

1. Responsive Video Using HTML5 <video>

Flexible Video Size

Make videos resize according to screen width using CSS.

```
<video controls class="video">
  <source src="movie.mp4" type="video/mp4">
</video>
.video {
  width: 100%;
  height: auto;
}
```

2. Using Aspect Ratio (Recommended Method)

To maintain proper video proportions:

```
.video-container {
  position: relative;
  padding-top: 56.25%; /* 16:9 aspect ratio */
}

.video-container video {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
```

Q. Explain Optimizing multimedia content for performance and accessibility

Ans.

Multimedia content such as **images, audio, and video** can greatly enhance a website, but if not optimized, it can **slow down performance** and reduce accessibility. Proper optimization ensures **faster loading, better user experience, and inclusivity**.

1. Performance Optimization

a) Compress Media Files

- Reduce file size without losing quality.
- Tools: **TinyPNG, ImageOptim, HandBrake (video)**
- Use modern formats: **WebP** (images), **MP4/WebM** (video), **OGG** (audio).

b) Use Responsive Media

- Serve different sizes for different devices using:
  - <picture> and srcset for images
  - Media queries for video size

2. Accessibility Optimization

a) Provide Alternative Text

- Use alt for images and descriptive text for icons.

```
<img src="logo.png" alt="Company Logo">
```

b) Captions and Transcripts

- Provide **captions for videos** and transcripts for audio.

```
<video controls>
  <source src="movie.mp4" type="video/mp4">
  <track src="captions.vtt" kind="captions"
srclang="en" label="English">
</video>
```

c) Keyboard & Screen Reader Support

- Ensure media controls can be accessed via keyboard
- Include **ARIA labels** when necessary.

d) Avoid Flashing Content

- Prevent seizures or discomfort by avoiding rapidly flashing videos or animations
- Optimizing multimedia content improves **loading speed, user experience, SEO, and inclusivity**, making websites fast, usable, and accessible to everyone.

Q. Explain Web Forms: Creating and handling user input forms for data collection.

Ans.

1. Introduction

Web forms are HTML elements used to **collect data from users** on websites. They allow users to input information such as text, selections, files, and submit it to a server for processing.

Uses:

- Contact forms
- Registration and login forms
- Surveys and feedback
- E-commerce checkout

Form Elements

Element	Description
<form>	Container for all input elements
<input>	Text, password, checkbox, radio, email, number, etc.
<textarea>	Multiline text input
<select>	Dropdown list
<option>	Options inside <select>
<button> / <input type="submit">	Submit the form
<label>	Label for input fields

Example: Registration Form (HTML + PHP)

HTML Form (index.html):

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
</head>
<body>
  <h2>User Registration Form</h2>
  <form action="submit.php" method="POST">
    <label for="name">Full Name:</label><br>
    <input type="text" id="name" name="name"
placeholder="Enter full name" required><br><br>

    <label for="email">Email:</label><br>
    <input type="email" id="email" name="email"
placeholder="Enter email" required><br><br>

    <label for="password">Password:</label><br>
```

```
    <input type="password" id="password"
name="password" placeholder="Enter password"
required><br><br>

    <label for="gender">Gender:</label><br>
    <input type="radio" id="male" name="gender"
value="Male">
    <label for="male">Male</label>
    <input type="radio" id="female"
name="gender" value="Female">
    <label for="female">Female</label><br><br>

    <label for="hobby">Hobbies:</label><br>
    <input type="checkbox" name="hobby[]"
value="Reading">Reading
    <input type="checkbox" name="hobby[]"
value="Traveling">Traveling
    <input type="checkbox" name="hobby[]"
value="Sports">Sports<br><br>

    <label for="country">Country:</label><br>
    <select name="country" id="country">
      <option value="India">India</option>
      <option value="USA">USA</option>
      <option value="UK">UK</option>
    </select><br><br>

    <input type="submit" value="Register">
  </form>
</body>
</html>
```

PHP Script to Handle Data (submit.php):

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $name = $_POST['name'];
  $email = $_POST['email'];
  $password = $_POST['password'];
  $gender = $_POST['gender'];
  $hobbies = isset($_POST['hobby']) ? implode(", ",
$_POST['hobby']) : '';
  $country = $_POST['country'];

  echo "<h2>Submitted Data</h2>";
  echo "Name: $name<br>";
  echo "Email: $email<br>";
  echo "Password: $password<br>";
  echo "Gender: $gender<br>";
  echo "Hobbies: $hobbies<br>";
  echo "Country: $country<br>";
}
?>
```

Q. Explain Responsive Typography, Principles, and implementing fluid typography with CSS techniques

Ans.

Responsive typography goes beyond simply resizing text at breakpoints. It ensures that:

- Text is **readable** on small and large screens
- Line length and spacing remain comfortable
- Visual hierarchy (headings, body, captions) stays clear
- Typography responds to **viewport size**, **user settings**, and **device capabilities**

It typically combines:

- Relative units (em, rem, vw)
- Fluid scaling
- Media queries
-

Core Principles of Responsive Typography

Readability First

Key factors:

- **Font size**: Body text usually 16–20px equivalent
- **Line length**: Ideal is 45–75 characters per line
- **Line height**: 1.4–1.7 for body text
- **Contrast**: Sufficient contrast between text and background

Hierarchy & Scale

Use a **typographic scale** so headings, subheadings, and body text relate logically.

Example scale:

- Body: 1rem
- H4: 1.25rem
- H3: 1.563rem
- H2: 1.953rem
- H1: 2.441rem

2.3 Flexibility Over Fixed Sizes

Avoid fixed px values for font sizing:

- Users can zoom
- Different devices render pixels differently
- Accessibility tools rely on scaling

Prefer:

- rem (root-relative)
- em (context-relative)
- vw (viewport-relative)

CSS Units Used in Responsive Typography

Unit	Use Case
px	Rarely (borders, hairline text)
em	Scales relative to parent
rem	Scales relative to root (html)
%	Inherited scaling
vw	Fluid scaling based on viewport
clamp()	Controlled fluid scaling

Implementing Fluid Typography in CSS

4.1 Fluid Typography with Viewport Units (Basic)

```
body {
 font-size: 2vw;
}
```

⚠️ **Problems**:

- Too small on mobile
- Too large on ultra-wide screens
- No minimum or maximum size

Fluid Typography with Media Queries (Classic Approach)

```
body {
 font-size: 16px;
}
@media (min-width: 768px) {
 body {
  font-size: 18px;
 }
}
@media (min-width: 1200px) {
 body {
  font-size: 20px;
 }
}
```

Fluid Type Scale Using CSS Variables

```
:root {
 --step--1: clamp(0.875rem, 0.83rem + 0.25vw,
1rem);
 --step-0: clamp(1rem, 0.95rem + 0.3vw, 1.125rem);
 --step-1: clamp(1.25rem, 1.15rem + 0.6vw, 1.5rem);
 --step-2: clamp(1.563rem, 1.4rem + 1vw,
1.953rem);
 --step-3: clamp(1.953rem, 1.7rem + 1.5vw,
2.441rem);
}

body {
 font-size: var(--step-0);
```

Q. Explain Fluid layout techniques.

Ans.

What Is a Fluid Layout?

A fluid layout:

- Uses **relative units** instead of fixed dimensions
- Adjusts continuously to viewport size
- Minimizes sudden layout "jumps"
- Works across phones, tablets, laptops, and large displays

It contrasts with:

- **Fixed layouts** (pixel-based)
- **Adaptive layouts** (preset breakpoints only)

Most modern responsive designs are **hybrid**: fluid by default, with breakpoints for refinement.

Core Principles of Fluid Layouts

2.1 Relative Sizing

Avoid fixed pixel dimensions for layout.

Use:

- %
- fr
- vw / vh
- rem / em
- auto

This allows content to scale naturally.

Content-Driven Design

Let content define layout constraints:

- Use intrinsic sizes (min-content, max-content)
- Avoid hard-coded heights
- Allow wrapping and reflow

Constraints for Readability

Pure fluidity can break usability. Always apply:

- max-width
- min-width
- clamp()

Fluid ≠ infinite scaling.

Q. Explain Testing Responsive typography on multiple devices and screen sizes.

Ans.

Testing responsive typography ensures that text remains readable, accessible, and visually consistent across different devices, screen sizes, and user settings. Because typography is affected by viewport size, pixel density, fonts, and OS preferences, it must be tested beyond simple browser resizing.

Why Testing Responsive Typography Matters

Without proper testing, you may encounter:

- Text that's too small on mobile
- Overly large headings on wide screens
- Broken hierarchy at certain widths

What to Test in Responsive Typography

2.1 Font Size & Scaling

Check:

- Minimum readable font size on small screens
- Maximum size on large monitors
- Smooth scaling between sizes (no sudden jumps)

Testing Across Screen Sizes (Viewport-Based)

Browser Responsive Mode (First Pass)

Use built-in tools:

- Chrome DevTools → Device Toolbar
- Firefox Responsive Design Mode
- Safari Responsive Design Mode

3.2 Continuous Resize Testing (Fluid Typography)

Resize the browser slowly and look for:

- Sudden font-size jumps
- Odd scaling with vw
- Clamp min/max limits being hit too soon

This is crucial when using clamp().

Best Workflow for Testing Responsive Typography

1. Start with **fluid type (clamp())**
2. Test continuously in browser resize
3. Test on real phones and tablets
4. Test zoom & OS text scaling
5. Test extreme widths
6. Fix issues with min/max constraints

Q. Explain How to Download & Install CodeIgniter + Composer Folder.

Ans.

Step 1: Install Composer

- Download Composer from **getcomposer.org**
- Install it on your system
- Verify installation using:

composer --version

Step 2: Create a Project Folder

- Open Command Prompt / Terminal
- Move to your web server directory (e.g., htdocs or www)

cd htdocs

Step 3: Download CodeIgniter Using Composer

Run the following command:

composer create-project codeigniter4/appstarter myproject

- codeigniter4/appstarter → Official starter package
- myproject → Project folder name

Composer downloads all required files and creates the CodeIgniter folder structure.

Step 4: Configure the Project

- Open the project folder
- Rename .env file and set:

app.baseURL = 'http://localhost/myproject/public'

Step 5: Start the Development Server

Run:

php spark serve

- Open browser and visit:

http://localhost:8080

Step 6: Understand Composer Folder

- The **vendor/** folder is created by Composer
- It contains:
    - CodeIgniter core files
    - Third-party libraries
- This folder should **not be edited manually**

Step 7: Installation Complete

- CodeIgniter is now successfully installed and ready for development.

Using Composer ensures a faster, secure, and well-managed installation of CodeIgniter. It automatically downloads dependencies, creates project structure, and simplifies updates.

Q. Explain File & Directory Structure

Ans.

File & Directory Structure of CodeIgniter

CodeIgniter follows the **MVC (Model–View–Controller)** architecture. Its directory structure helps organize code efficiently and improves maintainability, security, and scalability.

1. app/ Directory

This folder contains the **application logic**.

Important Subfolders:

- **Controllers/**
  Contains controller classes that handle user requests.
- **Models/**
  Handles database operations.
- **Views/**
  Stores HTML/PHP files used for UI display.
- **Config/**
  Configuration files (database, routes, app settings).

2. public/ Directory

- Serves as the **web root**
- Contains:
    - index.php (entry point of application)
    - CSS, JavaScript, images

✓ Improves security by preventing direct access to system files.

3. .env File

- Stores **environment-specific configurations**
- Examples:
    - Database credentials
    - Base URL
    - Environment mode

Helps separate configuration from code.

4. spark File

- Command-line interface for CodeIgniter
- Used to:
    - Run server
    - Create controllers/models
    - Run migrations

Example:

php spark serve

Q. Explain MVC Framework

Ans.

MVC is a software architectural pattern used in web application development. It divides an application into three interconnected components: Model, View, and Controller, which helps organize code and improve maintainability.

1. Model

- Handles **data and business logic**
- Communicates with the database
- Performs data validation and processing
  Example: fetching, inserting, updating records

2. View

- Represents the **user interface**
- Displays data to the user
- Contains HTML, CSS, and minimal PHP logic
  Example: web pages and templates

3. Controller

- **Acts as an intermediary between Model and View**
- **Receives user requests**
- **Processes input and decides which model and view to use**
  Example: handling form submissions

Working of MVC

1. User sends a request
2. Controller receives the request
3. Controller interacts with the Model
4. Model returns data
5. Controller loads the View with data

Advantages of MVC

- Clear separation of concerns
- Easy maintenance and scalability
- Supports parallel development
- Reusable and organized code

MVC framework improves application structure by separating data handling, business logic, and presentation, making development faster and applications more reliable.

Q. Explain Controller & views

Ans,

Controller

A **Controller** acts as the **brain of the application**.
It receives user requests, processes them, and decides how the application should respond.

Functions of Controller:

- Handles user input (URL, forms)
- Communicates with the Model
- Sends data to the View
- Controls application flow

Example: Handling a login request or form submission.

View

A **View** is responsible for the **presentation layer** of the application.
It displays data received from the Controller in a user-friendly format.

Functions of View:

- Displays UI (HTML/CSS)
- Shows dynamic data
- Contains minimal logic only for display

Example: Web pages, templates, dashboards.

Relationship Between Controller & View

- Controller processes requests and prepares data
- View displays that data to the user
- View does not directly access the Model

Advantages

- Clear separation of logic and presentation
- Easy maintenance and readability
- Better code reusability

Controllers manage application logic and user requests, while Views handle data presentation. Together, they ensure a structured and maintainable MVC application.

Q. Explain Routing Routes & Form

Ans.

Routing is the process of mapping a user's URL request to a specific **controller and method**. It decides which part of the application should handle the request.

Purpose of Routing:

- Controls application flow
- Creates clean, user-friendly URLs

Routes

Routes are the rules defined for routing. They specify **which controller method** should execute for a given URL.

Example:

- URL: /login
- Route maps it to: AuthController::login()

Benefits of Routes:

- Better URL structure
- Easy request handling

Form

A **Form** is used to **collect user input** such as text, passwords, email, etc., and submit it to the server.

Form Handling Process:

1. User fills the form (View)
2. Form data is submitted
3. Controller receives and validates data
4. Data is processed or stored using Model

Example

```
// app/Config/Routes.php
$routes->get('/', 'Home::index');
$routes->post('submit', 'Home::submit');
// app/Controllers/Home.php
class Home extends BaseController {
   public function index() {
      return view('form');
   }
   public function submit() {
      echo $this->request->getPost('name');
   }
}
<!-- app/Views/form.php -->
<form method="post" action="/submit">
  <input name="name">
  <button>Submit</button>
</form>
```

Q. Explain File handling with example

Ans.

File handling is used to create, read, write, and delete files stored on the server. PHP provides built-in functions to manage files efficiently.

Common File Handling Functions

Function	Purpose
fopen()	Open a file
fread()	Read file data
fwrite()	Write data to file
fclose()	Close file
file_exists()	Check file existence
unlink()	Delete file

File Open Modes

Mode	Description
r	Read only
w	Write (overwrite)
a	Append
r+	Read & write

Example: Writing and Reading a File

```
<?php
$file = fopen("data.txt", "w");
fwrite($file, "Hello File Handling");
fclose($file);

$file = fopen("data.txt", "r");
echo fread($file, filesize("data.txt"));
fclose($file);
?>
```

Explanation of Example

1. fopen() opens the file
2. fwrite() writes data
3. fread() reads data from file
4. fclose() closes the file

Advantages of File Handling

- Stores data permanently
- Easy data management
- Useful for logs and reports

File handling in PHP allows efficient management of server files by using simple built-in functions to read, write, and manipulate file data.

Q. Explain Sending email with example

Ans.

Sending email allows a web application to **communicate with users** by sending notifications, confirmations, alerts, or reports. PHP provides the built-in mail() function and supports SMTP-based emailing for better reliability.

Basic Requirements

- A web server with mail support
- Correct sender and recipient email addresses
- SMTP configuration (recommended)

PHP mail() Function Syntax

mail(to, subject, message, headers);

Example: Sending Email Using PHP

```
<?php
$to = "user@example.com";
$subject = "Test Mail";
$message = "This is a test email.";
$headers = "From: admin@example.com";

if(mail($to, $subject, $message, $headers)) {
   echo "Email sent successfully";
} else {
   echo "Email sending failed";
}
?>
```

Explanation of Example

- $to → Recipient email address
- $subject → Email subject
- $message → Email body
- $headers → Sender details
- mail() → Sends the email

Advantages of Sending Email

- User notifications
- Account verification
- Password recovery
- System alerts

Q. Explain Cookie and Session

Ans.

A cookie is a small piece of data stored on the user's browser by the server. It is used to remember user information for future visits.

Features of Cookies

- Stored on client side
- Limited size (about 4KB)
- Can have expiration time
- Less secure

Example: Cookie in PHP

```
<?php
setcookie("user", "John", time()+3600);
echo $_COOKIE["user"];
?>
```

Explanation

- setcookie() creates a cookie named **user**
- Stored for 1 hour
- Data can be accessed using $_COOKIE

Session

A **session** stores user data on the **server** and assigns a unique session ID to the user.

Features of Sessions

- Stored on server side
- More secure than cookies
- Data is destroyed after session ends
- Used for login systems

Example: Session in PHP

```
<?php
session_start();
$_SESSION["user"] = "John";
echo $_SESSION["user"];
?>
```

Explanation

- session_start() starts session
- Session variable stores user data
- Data is available until session ends

Difference Between Cookie and Session

Cookie	Session
Stored on client	Stored on server
Less secure	More secure
Has expiry time	Ends on browser close
Small storage	Larger storage

Q. Explain Restful and Restless API integration with example

Ans.

API (Application Programming Interface) integration allows one application to communicate with another application to exchange data or services.

RESTful API

RESTful API follows the **REST (Representational State Transfer)** architectural principles. It uses standard HTTP methods and resources identified by URLs.

Key Features of RESTful API

- Uses HTTP methods: **GET, POST, PUT, DELETE**
- Stateless communication
- Resource-based URLs
- Data usually in **JSON or XML**
- Easy to scale and maintain

RESTful API Example

Request

GET /api/users/1

Response (JSON)

```
{
 "id": 1,
 "name": "John",
 "email": "john@example.com"
}
```

Explanation

- URL represents a resource (users)
- GET method fetches data
- Clean and meaningful structure

RESTless API

RESTless API does **not follow REST principles strictly**. It behaves like **RPC (Remote Procedure Call)** where URLs represent actions instead of resources.

Key Features of RESTless API

- Action-based URLs
- Mostly uses **POST method**
- Less standardized
- Harder to scale and maintain

RESTless API Example

Request

POST /api/getUser

Request Body

json

Copy code

```
{
 "id": 1
}
```

Response

json

Copy code

```
{
 "name": "John",
 "email": "john@example.com"
}
```

Explanation

- URL represents an action (getUser)
- No clear use of HTTP methods
- Business logic tied to endpoint name

Integration Example (PHP – RESTful)

```
$response = file_get_contents("https://api.example.com/users/1");
$data = json_decode($response, true);
echo $data['name'];
```

Differences Between RESTful and RESTless API

RESTful API	**RESTless API**
Resource-based	Action-based
Uses HTTP methods properly	Mostly uses POST
Stateless	Often stateful
Scalable & standard	Less scalable
Clean URLs	Complex URLs

Advantages of RESTful API

- Platform independent
- Better performance
- Easy integration
- Widely supported

RESTful API integration follows standard web principles using HTTP methods and resource-based URLs, making it efficient and scalable. RESTless APIs lack these standards and are harder to maintain. Therefore, RESTful APIs are preferred for modern web applications.

RESTful APIs follow standard HTTP methods and resource-based architecture, while RESTless APIs use action-based endpoints without strict REST rules.

Q. Explain MySQL, CRUD operation with MySQL

Ans.

MySQL

MySQL is an **open-source relational database management system (RDBMS)** used to store, manage, and retrieve data efficiently. It uses **Structured Query Language (SQL)** and is widely used with PHP for web applications.

Features of MySQL

- Open source and fast
- Uses tables (rows and columns)
- Supports large databases
- Secure and reliable
- Easy integration with PHP

CRUD Operations in MySQL

CRUD stands for **Create, Read, Update, and Delete**. These are the basic operations performed on a database.

Database & Table Example

```
CREATE DATABASE college;
USE college;

CREATE TABLE students (
 id INT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(50),
 email VARCHAR(50)
);
```

1. Create (Insert Data)

```
INSERT INTO students (name, email)
VALUES ('Rahul', 'rahul@gmail.com');
```

Purpose: Adds new records to the table.

2. Read (Fetch Data)

```
SELECT * FROM students;
```

Purpose: Retrieves data from the table.

3. Update (Modify Data)

```
UPDATE students
SET email = 'rahul123@gmail.com'
WHERE id = 1;
```

Purpose: Updates existing records.

4. Delete (Remove Data)

```
DELETE FROM students WHERE id = 1;
```

Purpose: Deletes records from the table.

CRUD Example Using PHP + MySQL

```
<?php
$conn =
mysqli_connect("localhost","root","","college");

// Insert
mysqli_query($conn,"INSERT INTO
students(name,email)
VALUES('Amit','amit@gmail.com')");

// Read
$result = mysqli_query($conn,"SELECT * FROM
students");

// Update
mysqli_query($conn,"UPDATE students SET
name='Amit Kumar' WHERE id=1");

// Delete
mysqli_query($conn,"DELETE FROM students
WHERE id=1");
?>
```

Advantages of CRUD Operations

- **Easy data management**
- **Efficient data handling**
- **Supports dynamic applications**

MySQL is a powerful database system used for storing structured data. CRUD operations allow developers to create, read, update, and delete records, forming the foundation of database-driven applications.

MySQL is a relational database system where CRUD operations are used to manage data using SQL commands.

Q. Explain Query builder with example

Ans.

Query Builder is a **database abstraction tool** provided by CodeIgniter (or other frameworks) to **create SQL queries programmatically**.

It allows developers to **build queries safely, easily, and efficiently** without writing raw SQL.

Advantages of Query Builder

- Prevents SQL injection
- Easier to read and maintain
- Supports chaining of methods
- Works with multiple database types

Common Query Builder Methods

Method	Purpose
select()	Select columns
from()	Specify table
where()	Add conditions
get()	Execute select query
insert()	Insert data
update()	Update data
delete()	Delete data

Example 1: Select Data

```
<?php
$db = \Config\Database::connect();

$builder = $db->table('students');
$builder->select('id, name, email');
$builder->where('id', 1);
$query = $builder->get();

$result = $query->getResult();
print_r($result);
?>
```

Explanation:

- table('students') → selects table
- select('id, name, email') → selects columns
- where('id', 1) → adds condition
- get() → executes the query

Example 2: Insert Data

```
$data = [
    'name' => 'Amit',
    'email' => 'amit@gmail.com'
];

$builder = $db->table('students');
$builder->insert($data);
```

Example 3: Update Data

```
$builder = $db->table('students');
$builder->set('email', 'amit123@gmail.com');
$builder->where('id', 1);
$builder->update();
```

Example 4: Delete Data

```
$builder = $db->table('students');
$builder->where('id', 1);
$builder->delete();
```

Query Builder simplifies database operations by providing **method-based syntax** instead of raw SQL. It improves security, readability, and maintainability of database-driven applications.

Query Builder in CodeIgniter allows building database queries safely and efficiently using methods like select, insert, update, and delete instead of raw SQL.

Q. Explain Deployment with example

Ans.

Deployment is the process of making a web application **live on a server** so that users can access it over the internet. It involves transferring code, databases, and assets from the local development environment to a production server.

Steps for Deployment

1. **Prepare the Application**
    - Remove debugging tools
    - Set environment to production in CodeIgniter
    - Ensure all configurations are correct (.env, database, baseURL)
2. **Choose a Hosting Platform**
    - Shared hosting, VPS, or cloud hosting (AWS, DigitalOcean, etc.)
    - Ensure PHP and MySQL are installed
3. **Upload Files**
    - Copy project files (except sensitive files like writable/logs if not needed)
    - Keep **public/** folder as web root
4. **Configure Database**
    - Export local database using phpMyAdmin or mysqldump
    - Import it on the production server
5. **Update Configuration**
    - Update .env or Config/App.php with production database, baseURL, and environment
6. **Set File Permissions**
    - Ensure writable/ folder has write permissions
7. **Test the Application**
    - Visit your domain
    - Check pages, forms, database connections, and emails

Example: Deploying CodeIgniter to cPanel Hosting

1. Zip your CodeIgniter project
2. Upload it via **File Manager** or FTP
3. Extract files, set **public/** as root folder
4. Import the database through phpMyAdmin
5. Update .env with:

database.default.hostname = localhost
database.default.database = mydb
database.default.username = user
database.default.password = pass
app.baseURL = 'https://mydomain.com/'

6. Set writable/ folder permissions to **755 or 775**
7. Visit https://mydomain.com/ to verify deployment

Advantages of Deployment

- Makes application accessible to users
- Enables testing in production environment
- Supports client use and feedback

Deployment is the process of moving a web application from local development to a live server, including code, database, and configuration setup.

Q. Hosting (AWS/Hostinger/Google Cloud) with example.

Ans.

Hosting is the process of storing a website or web application on a **server** so that it is accessible over the internet via a domain name. Hosting providers provide space, server resources, and network connectivity.

Types of Hosting

1. **Shared Hosting** – Multiple websites share the same server (cheap, limited control)
2. **VPS Hosting** – Virtual private server with more control and resources
3. **Cloud Hosting** – Scalable hosting using multiple cloud servers

Popular Hosting Platforms

1. AWS (Amazon Web Services)

- Cloud-based hosting service
- Provides **EC2** servers, S3 storage, RDS database, and more
- Suitable for scalable applications
- Example: Hosting a PHP website

- Launch **EC2 instance**
- Upload project via SSH/SFTP
- Install Apache, PHP, MySQL
- Set domain DNS to EC2 IP

2. Hostinger

- Affordable shared hosting provider
- Easy-to-use cPanel interface
- Provides PHP, MySQL, and Email hosting
- Example: Deploying CodeIgniter
  - Upload files via **File Manager / FTP**
  - Import database using phpMyAdmin
  - Update .env or Config/App.php
  - Set public folder as root

3. Google Cloud Platform (GCP)

- Cloud-based scalable hosting like AWS
- Provides Compute Engine (VM), Cloud SQL (Database), App Engine
- Example: Hosting Laravel or CodeIgniter
  - Deploy project on **Compute Engine VM**
  - Configure Apache/Nginx
  - Upload files and database
  - Set domain DNS to VM external IP

Advantages of Hosting

- Makes website accessible globally
- Provides secure and reliable servers
- Supports large-scale and high-traffic applications
- Offers backups, email services, and domain management

Example Comparison

Provider	Type	Key Feature	Example Use
AWS	Cloud	Highly scalable, pay-as-you-go	Hosting dynamic web app
Hostinger	Shared	Affordable, beginner-friendly	Simple PHP/CodeIgniter site
GCP	Cloud	Enterprise-grade, flexible resources	Large-scale web apps

Hosting is storing a website on a server so it is accessible online; AWS, Hostinger, and Google Cloud provide different hosting solutions for various needs